Recover the Design

Date: 2026-10-23

Abstract

We recover a fence layout from connector degrees. Candidate edges join neighboring connectors in each row and column. We reject impossible degree totals, then express the remaining degree and crossing rules as Boolean clauses. A self-contained C++17 solver uses watched literals, conflict learning and backtracking to find a valid assignment. The current version passes 968 grid tests and 2,000 independent Boolean tests under both release and sanitizer builds. Learning removes repeated failed choices, but worst-case search and storage can still grow exponentially.

Contents

1 Introduction **3**

2 Data Structure / Algorithm Specification **3**

2.1 Candidate Edges 3

2.2 Component Checks and Boolean Encoding 3

2.3 Propagation, Learning and Backtracking 4

2.4 Correctness Argument 4

3 Testing Results **5**

3.1 Environment and Method 5

3.2 Correctness Tests 5

3.3 Timing Results 6

4 Analysis and Comments **7**

4.1 Time Complexity 7

4.2 Space Complexity 7

4.3 Limitations and Improvements 7

5 Conclusion **8**

6 References **8**

7 Source Code Appendix **8**

1 Introduction

According to the PTA problem Recover the Design, each input contains one $n \times m$ grid, where $1 < n, m \leq 50$. A value from 1 to 4 gives a connector's required degree; zero means no connector. Fences are horizontal or vertical. They can turn or cross only at connectors.

For each connector, output `row col up down left right` in row order, from left to right. Coordinates start at 1; direction flags are 0 or 1. Separate numbers with one space, with no leading or trailing spaces. Print `No Solution` if no layout exists. Any valid layout is accepted. Separate components and cycles are allowed; an empty grid produces no output.

2 Data Structure / Algorithm Specification

2.1 Candidate Edges

We model the layout as a graph: each vertex represents a connector, and each candidate edge joins consecutive connectors in the same row or column. This gives each vertex at most four edges. A longer fence is simply a sequence of these edges, so we do not need separate edges that skip connectors.

Let $x_e \in \{0, 1\}$ show whether edge e is selected. For a vertex v with required degree k_v, require $\sum_{e \ni v} x_e = k_v$. If two edges e and f cross at an empty cell, require $x_e + x_f \leq 1$.

To build this graph, we scan the rows and columns and store each edge's endpoints and direction indices. We also use two grids to record the horizontal and vertical edge through each empty cell. When both are present, the pair goes into the conflict lists.

2.2 Component Checks and Boolean Encoding

First, we check each connected component of the candidate graph. Its required degree sum must be even because every selected edge contributes two endpoints. If breadth-first coloring shows that the actual component is bipartite, both color classes must have equal required degree sums. We do not color by grid coordinates: a long edge can join two cells of the same coordinate parity. For nonbipartite components, only the even-sum check applies.

Each candidate edge is one Boolean variable. A clause is a list of alternatives, at least one of which must hold. At a connector with d candidate edges and required degree k, we reject $k > d$. Otherwise, for every subset of $k + 1$ edges, we require at least one edge to be excluded. For every subset of $d - k + 1$ edges, we require at least one to be selected. Together these clauses require exactly k selected edges. Since $d \leq 4$, each vertex produces only a constant number of short clauses. A crossing pair adds the clause $\neg x_e \vee \neg x_f$.

2.3 Propagation, Learning and Backtracking

Our Boolean engine follows standard conflict-driven clause learning [1]. We implemented the engine within the submitted source file; it does not link to an external SAT library. It uses two watched literals per non-unit clause, first-UIP conflict analysis, variable activity, saved choices and restarts. These are standard mechanisms; the graph encoding and component checks are specific to this problem.

When a watched alternative becomes false, we try to watch another alternative that is not false. If none exists, the other watch must become true; if it is already false, the clause gives a conflict. This updates affected clauses instead of repeatedly scanning the entire graph.

For each implication, we store its reason clause and decision level. After a conflict, we follow these reasons backward until only one literal in the learned clause belongs to the current decision level. We undo later assignments, keep earlier ones, and assert the learned clause's remaining literal. The clause stays in memory, so the search does not repeat the same conflicting combination. Variables involved in conflicts receive higher activity; decisions reuse their last assigned values.

After 100 conflicts, we restart from root assignments while keeping learned clauses. The interval grows by a factor of 1.5 up to 10,000 conflicts. This implementation retains all learned clauses. There is no solver time cutoff and no path that treats a timeout as an unsatisfiable input.

2.4 Correctness Argument

Every legal fence can be split at connectors into candidate edges, so the graph representation is complete. The component tests are necessary degree-sum identities and cannot discard a valid design. The subset clauses forbid both more than k selected edges and more than $d - k$ excluded edges. Thus they express exactly the required degree. Crossing clauses express exactly the forbidden empty-cell intersections.

Unit propagation assigns only a literal required by a clause. Conflict analysis resolves the conflict with stored reason clauses; resolution preserves logical consequence [1]. Therefore learned clauses exclude no valid design. Backtracking removes assignments rather than constraints, and restarts retain both original and learned clauses. The standard complete CDCL search applies to this finite encoding [1]. Our local exhaustive comparisons check the implementation separately from that general algorithmic argument.

When search succeeds, every variable is assigned and every clause is satisfied. We print matching directions at both endpoints of each selected edge, in input order. A root-level contradiction proves the encoding unsatisfiable. Therefore the output is a valid layout or the required `No Solution`.

3 Testing Results

3.1 Environment and Method

The original tests and timing table were collected on 2026-10-21 using WSL2, an AMD Ryzen 7 H 255 CPU, GCC 13.3.0 and Python 3.13.13. After the search change, we reran correctness tests on 2026-11-01 using macOS 26.6 arm64, Apple Clang 21.0.0 and Python 3.9.6. The release build uses `-std=c++17 -O2 -Wall -Wextra -Wpedantic`; the second build enables ASan and UBSan. These are local automated results, not a record of independent member approval.

The output checker traces directions through the grid and checks degree, reciprocity, order, spacing and crossings. Small grid instances are also compared with exhaustive edge-subset enumeration. A separate harness compiles the actual Boolean engine and compares 2,000 seeded formulas of one to eight variables against complete truth tables, including satisfiable and unsatisfiable inputs. Returned assignments are checked against every input clause.

3.2 Correctness Tests

Test case	Count	Expected / actual result
Supplied sample	1	Valid design; PASS
All 2 by 2 degree grids	625	Match exhaustive check; PASS
Random 3 by 3 grids	200	Match exhaustive check; PASS
Crossing, junction, multiple solutions, straight-through	4	Expected outcomes; PASS
Generated valid 4 by 4 designs	100	Valid designs; PASS
Original structured 50 by 50 grids	5	Expected outcomes; PASS
Component and performance regressions	9	Expected outcomes within 2 seconds; PASS
Planted 20 by 20 and 50 by 50 designs	24	Valid designs within 2 seconds; PASS

Both builds pass all 968 grid cases, three invalid-output rejection checks and 2,000 Boolean formulas. No runtime sanitizer error was reported. The original grid generator uses seed 17; the Boolean harness uses seed 20261101. The 24 larger cases use sizes 20 and 50, layout densities 0.25, 0.6 and

1.0, and seeds 17 to 20. We plant noncrossing edges first, so feasibility is known independently of the solver.

The nine regressions include odd degree totals, unequal bipartite totals, two separate odd components with an even combined total, and feasible long-edge and odd-cycle layouts. They protect against both missing pruning and incorrect coordinate-based coloring.

The earlier `rollback-a` and `rollback-b` inputs remain legal, but the new engine solves them without a conflict. The updated instrumentation instead uses the planted 50 by 50 case with density 1.0 and seed 17. It records 412 conflicts, 412 learned clauses, 414 effective backtracks, 104,093 undone assignments, 280 jumps across multiple decision levels, and two restarts. `scripts/check_rollback.py` reproduces these events on a temporary copy without changing decisions. These counters measure distinct events; undone assignments are not a count of backtracks.

3.3 Timing Results

The following table is preserved from the original 2026-10-21 solver. It is historical data, not a measurement of the new learning engine. Each entry used one warmup and ten runs and includes process startup, input/output, solving and Python validation.

Input	Median (ms)	Range (ms)
Empty grid	1.868	1.392 to 2.131
Degree 1 everywhere	41.276	38.732 to 42.976
All neighboring edges required	8.283	7.785 to 10.294
Degree 1 at both ends of each row	2.292	1.932 to 2.749
Degree 4 everywhere (no solution)	2.520	2.085 to 2.909

The original solver exceeded 15 seconds on a 9 by 9 all-one grid, which has the impossible odd degree sum 81. It took about 10.701 seconds on a 10 by 10 all-one grid with positions (1,1) and (1,3) changed to 2; the two candidate-graph color classes require totals 52 and 50. The latter case has an even combined total, so a parity check alone is insufficient.

On the current local machine, ten runs after one warmup gave new end-to-end medians of 1.663 ms and 1.528 ms for those two cases. The 49 by 49 odd-sum and 50 by 50 unequal-color-total cases had medians of 2.707 ms and 2.796 ms. These measurements use the same timing boundary as the earlier table, but the hardware and compiler differ from that historical table. The four dense

planted 50 by 50 cases completed in about 39–58 ms during the expanded release check. These cases improve coverage; they do not establish a worst-case time bound or PTA acceptance.

4 Analysis and Comments

4.1 Time Complexity

Let V count connectors, E candidate edges and X crossing pairs. Each connector has at most four candidate edges, so $E \leq 2V$. Each empty cell contributes at most one crossing pair, so $X \leq nm$. Building the graph, checking its components and generating the initial clauses takes $O(nm + V + E + X)$ time. The degree encoding has constant size per connector because $d \leq 4$.

For the search, let K be the number of conflicts and L the peak total number of stored clause literals, including learned clauses. Between conflicts or restarts, at most E variables are assigned. A conservative bound for propagation is $O(EL)$ for that interval; scanning variables for each decision costs $O(E^2)$. Conflict analysis visits at most E reasons, each of length at most E. Since restarts occur only after conflicts, a conservative bound for the observed search is $O((K+1)E(E+L))$, in addition to graph construction. This bound is derived from our loops, not a polynomial guarantee: K and learned storage can grow exponentially [1].

4.2 Space Complexity

Graph and grid storage is $O(nm + V + E + X)$. Clauses and their watch lists add $O(L)$ space. Assignments, reasons, decision levels, activity and the trail use $O(E)$ auxiliary space; the initial graph traversal also uses $O(V)$. Total space is $O(nm + V + E + X + L)$. Retaining all learned clauses trades memory for avoiding repeated failed searches; it does not preserve the old solver's linear total-space bound.

4.3 Limitations and Improvements

1. Search remains exponential in the worst case. The recorded statement has no time or memory limits, and PTA acceptance has not been checked.
2. Learned clauses are retained. A carefully tested clause-removal policy could bound practical memory growth while preserving active reasons.
3. Larger planted tests cover feasible layouts, but they do not represent every difficult sparse or nearly unsatisfiable instance.

5 Conclusion

Component checks reject simple global contradictions before search. Clause learning and watched propagation then avoid repeated failed combinations and full-graph scans. The observed slow cases are fixed, while the report keeps the limits of local testing explicit.

6 References

[1] N. Eén and N. Sörensson, "An Extensible SAT-solver," in *Theory and Applications of Satisfiability Testing*, LNCS, vol. 2919. Berlin, Heidelberg, Germany: Springer, 2004, pp. 502–518. doi:10.1007/978-3-540-24605-3_37.

7 Source Code Appendix

The solver is in `code/main.cpp`, the checker in `scripts/check.py`, and the sample in `tests/sample1.in`. See `README.md` for build and test commands.